CLIENT-EDGE ARCHITECTURE • ENGINEERING GUIDE

Backend for Frontend (BFF) Pattern in Microservices

Decomposition of Multi-Client Discrepancies, Gateway Bottlenecks, Payload Optimization, and Team Ownership

EXECUTIVE SUMMARY

Modern applications serve radically divergent client environments: high-bandwidth desktop browsers, battery-constrained mobile radios, and screenless voice IoT devices. Generic single API gateways create organizational bottlenecks and payload bloat. The Backend for Frontend (BFF) pattern assigns each client type a dedicated, frontend-owned backend aggregator service.

1. THE MULTI-CLIENT CONSTRAINT SPECTRUM

1. Desktop Web

Unbounded screen real estate; high bandwidth. Demands large, rich aggregations (50 products with reviews).

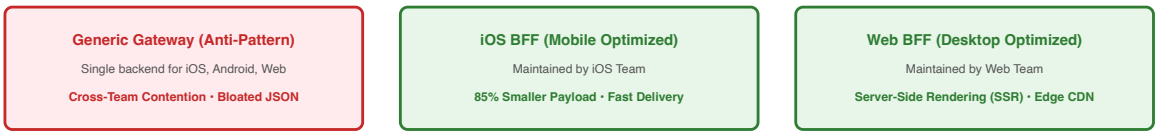
2. Mobile (iOS / Android)

Cellular latency, battery drain. Requires lean payloads, aggressive compression, and minimal roundtrips.

3. Smart Wearables / IoT

Low-power Bluetooth, tiny screens. Demands single-attribute status responses ("Out for Delivery").

2. GENERIC GATEWAY BOTTLENECK VS. BFF ARCHITECTURE



3. CONCRETE PAYLOAD COMPARISON: MOBILE VS. DESKTOP

Desktop Web BFF Response (42 KB)

```
{
  "orderId": "ORD-99",
  "total": "$142.50",
  "items": [
    {"name": "Headphones", "specs": {...}, "warranty":
{...}}
  ],
  "fullTrackingHistory": [... 25 hops ...],
  "customerProfile": {"taxId": "...", "preferences":
{...}}
}
```

Mobile App BFF Response (3 KB)

```
{
  "id": "ORD-99",
  "total": "$142.50",
  "statusText": "Arriving today by 5 PM",
  "trackingProgressBar": 75,
  "heroThumbnailUrl": "https://cdn.internal/h.webp"
}
```

85% network reduction, saving mobile radio battery and eliminating 5 sequential requests.

4. ORGANIZATIONAL OWNERSHIP (CONWAY'S LAW ALIGNMENT)

The defining rule of the BFF pattern is that **the client squad owns the BFF service**:

Component	Owning Team	Deployment Cycle
Mobile BFF	iOS & Android Frontend Engineers	Tied directly to mobile feature releases.
Web BFF	React / Next.js Web Squad	Continuous deployment (multiple times/day).
Domain Microservices	Backend Core Domain Teams (Orders, Payments)	Strict API contracts; decoupled from UI changes.

5. AVOIDING THE "FAT BFF" ANTI-PATTERN

A frequent failure mode is allowing domain business logic to leak into the BFF layer:

Permitted in BFF (Aggregation / Formatting)

- Parallel fan-out across downstream microservices via gRPC.
- Filtering out unwanted fields to minimize mobile payload bytes.
- Formatting dates and currency symbols for client locales.
- Translating cookies into backend JWT authorization headers.

Forbidden in BFF (Domain Leakage)

- Calculating shopping cart tax or applying discount rules.
- Executing direct database writes or managing business state.
- Enforcing business authorization rules (belongs in domain service).
- Direct database connection strings or shared SQL access.

6. PRODUCTION IMPLEMENTATION IN NODE.JS / TYPESCRIPT

```
// Mobile BFF Aggregation Endpoint (TypeScript / Express)
app.get("/api/mobile/order/:id", async (req, res) => {
    const traceCtx = extractTraceContext(req);
    // Parallel downstream fan-out over multiplexed gRPC
    const [order, shipment] = await Promise.all([
        orderClient.getOrder({ id: req.params.id }, traceCtx),
        shippingClient.getTracking({ id: req.params.id }, traceCtx)
    ]);
    // Shape compact mobile-tailored response
    res.json({
        id: order.id,
        total: formatCurrency(order.amountCents),
        status: shipment.currentMilestoneText,
        progress: shipment.percentComplete
    });
});
```

7. PRODUCTION BFF VERIFICATION CHECKLIST

BFF Architecture Standards

1. **No Core Business Logic:** If logic is shared between iOS and Web, it belongs in downstream domain services.
2. **Frontend Team On-Call:** Frontend squads must monitor their BFF's latency and error rate in PagerDuty.
3. **Downstream gRPC:** BFF must use gRPC over HTTP/2 to communicate with internal microservices.
4. **Strict Payload Benchmarks:** Continuously monitor mobile BFF payload size in CI/CD performance testing.